

User Guide — Construction Cost Estimation AI

End-to-end guide to set up and run the full stack locally (development) or in production.

Table of Contents

- 1. [Architecture Overview](#)
- 2. [Prerequisites](#)
- 3. [Clone the Repository](#)
- 4. [Environment Configuration](#)
- 5. [Install Dependencies](#)
- 6. [Development — Quick Start \(All-in-One\)](#)
- 7. [Development — Manual Start \(Step by Step\)](#)
- 8. [Database Migrations](#)
- 9. [Seed & Ingest BSR Data](#)
- 10. [Accessing the App](#)
- 11. [Running Tests](#)
- 12. [Production Deployment](#)
- 13. [Common Commands Reference](#)
- 14. [Troubleshooting](#)

1. Architecture Overview

Layer	Technology	Port
Frontend	Next.js (React)	3000
Backend API	FastAPI (Python)	8000
Database	PostgreSQL 16	5436
Vector Store	ChromaDB	8080

Layer	Technology	Port
Session Cache	Redis 7	6379

The backend uses a deterministic RAG pipeline to match Bill of Quantities (BOQ) items against Building Schedule of Rates (BSR) using PostgreSQL + ChromaDB. An Ollama-backed LLM handles project description understanding.

2. Prerequisites

Install all tools before proceeding:

Tool	Version	Purpose
Docker Desktop	v24+	Runs Postgres, Chroma, Redis
Python	3.10+	Backend runtime
Node.js	18+	Frontend runtime
Git	Any	Source control

Verify installations:

```
docker --version
python --version
node --version
npm --version
```

3. Clone the Repository

```
git clone <repository-url>
cd codebase
```

4. Environment Configuration

Two `.env.local` files are required — one per service.

4a. Backend — `backend/.env.local`

```
cp backend/.env.local.example backend/.env.local
```

Open `backend/.env.local` and fill in:

Key	Description
POSTGRES_USER	DB username (default: <code>user</code>)
POSTGRES_PASSWORD	DB password (default: <code>user1234</code>)
POSTGRES_DB	DB name (default: <code>estimate_db</code>)
OLLAMA_API_KEY	Your Ollama cloud API key
OLLAMA_MODEL	LLM model name (e.g. <code>llama3.2</code>)
OLLAMA_HOST	Ollama API endpoint
API_SECRET_KEY	Random secret for JWT signing
CORS_ALLOW_ORIGINS	<code>http://localhost:3000</code> for dev
SMTP_USERNAME	Gmail address for email sending
SMTP_PASSWORD	Gmail App Password (not account password)

Gmail App Password: Go to Google Account → Security → 2-Step Verification → App passwords.

4b. Frontend — `frontend/.env.local`

Create manually or copy from an example:

```
# frontend/.env.local
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
NEXT_PUBLIC_APP_URL=http://localhost:3000
AUTH_SECRET=<run: openssl rand -base64 32>
UPLOADTHING_TOKEN=<your uploadthing token>
```

Generate AUTH_SECRET : openssl rand -base64 32 (or use any random 32+ char string).

5. Install Dependencies

Backend

```
cd backend
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
cd ..
```

Frontend

```
cd frontend
npm install
cd ..
```

6. Development — Quick Start (All-in-One)

A single script starts everything: Docker infra, backend, frontend, migrations, seed, and BSR ingest.

macOS / Linux / Git Bash on Windows:

```
./dev.sh up
```

This will:

1. Start Postgres, ChromaDB, Redis via Docker
2. Wait for Postgres to be healthy
3. Run Alembic DB migrations
4. Seed categories into the database
5. Ingest BSR PDF data (skipped automatically if already done)
6. Start the FastAPI backend on `http://localhost:8000`
7. Start the Next.js frontend on `http://localhost:3000`

Press `ctrl+C` to stop the backend and frontend (Docker infra keeps running).

To stop the Docker infra:

```
./dev.sh down
```

7. Development — Manual Start (Step by Step)

Use this approach if the all-in-one script does not fit your workflow, or on Windows PowerShell.

Step 1 — Start Docker Infrastructure

```
docker compose --project-name estimation --env-file backend/.env.local -f backend/docker/docker-
```

Verify containers are running:

```
docker ps
```

Expected: `estimation_postgres` , `estimation_chroma` , `estimation_redis` all Up .

Step 2 — Run Database Migrations

```
cd backend

# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate

alembic upgrade head
```

Step 3 — Seed Categories

```
# (inside backend/, venv active)
python scripts/seed_categories.py
```

Step 4 — Ingest BSR Data

```
# (inside backend/, venv active)
python scripts/ingest_bsr.py
```

Re-running is safe — the script is idempotent and skips if data already exists. Pass `--force` to re-ingest.

Step 5 — Start the Backend

```
# (inside backend/, venv active)
uvicorn app.api.server:app --reload --host 0.0.0.0 --port 8000
```

Step 6 — Start the Frontend

Open a new terminal:

```
cd frontend
npm run dev
```

8. Database Migrations

Alembic manages the PostgreSQL schema. Run whenever you pull new changes.

```
cd backend
# venv active
alembic upgrade head      # apply all pending migrations
alembic current           # show current revision
alembic history           # list all revisions
```

Or via the dev script:

```
./dev.sh migrate
```

9. Seed & Ingest BSR Data

These steps are required once before using the estimation features.

```
# Seed work categories
./dev.sh seed

# Ingest bundled BSR PDF (bsr_wp_2025.pdf)
./dev.sh ingest

# Ingest a custom PDF
./dev.sh ingest /path/to/custom_bsr.pdf
```

10. Accessing the App

Service	URL
Frontend (Web App)	http://localhost:3000
Backend API	http://localhost:8000

Service	URL
API Docs (Swagger)	http://localhost:8000/docs
API Docs (ReDoc)	http://localhost:8000/redoc
Health Check	http://localhost:8000/healthz

11. Running Tests

Backend Tests

```
cd backend
# venv active
pytest                # run all tests
pytest -m "not integration" # unit tests only
pytest --cov           # with coverage report
```

Frontend Tests

```
cd frontend
npm test                # unit tests (Vitest)
npm run test:coverage   # with coverage
npm run test:e2e        # Playwright end-to-end tests
```

E2E tests require both the backend and frontend to be running (`./dev.sh up` first).

12. Production Deployment

Production uses a single Docker Compose file that builds all services as containers.

Step 1 — Create Production Environment File

Create `.env` at the repository root (not `.env.local`):


```
cp backend/.env.local.example .env
# Edit .env – set all values, especially:
#   API_SECRET_KEY=<strong random secret>
#   POSTGRES_PASSWORD=<strong password>
#   OLLAMA_API_KEY=<your key>
#   AUTH_SECRET=<run: openssl rand -base64 32>
#   NEXT_PUBLIC_API_BASE_URL=https://your-domain.com
```

Step 2 — Build and Start

```
./run.sh up          # build images and start all services
./run.sh status      # check running containers
./run.sh logs        # tail all logs
```

Step 3 — Run Migrations in Production

```
./run.sh migrate
```

Other Production Commands

```
./run.sh down        # stop all services
./run.sh rebuild      # rebuild images from scratch (no cache)
```

13. Common Commands Reference

dev.sh (Development)

Command	Description
./dev.sh up	Start everything (infra + backend + frontend)
./dev.sh down	Stop Docker infrastructure
./dev.sh infra	Start Docker infra only
./dev.sh backend	Start FastAPI backend only

Command	Description
<code>./dev.sh frontend</code>	Start Next.js frontend only
<code>./dev.sh migrate</code>	Run Alembic migrations
<code>./dev.sh seed</code>	Seed categories
<code>./dev.sh ingest</code>	Ingest BSR PDF
<code>./dev.sh logs</code>	Tail Docker logs
<code>./dev.sh status</code>	Show container status

run.sh (Production)

Command	Description
<code>./run.sh up</code>	Build images and start all services
<code>./run.sh down</code>	Stop all services
<code>./run.sh rebuild</code>	Rebuild all images (no cache)
<code>./run.sh migrate</code>	Run migrations in backend container
<code>./run.sh logs</code>	Tail all service logs
<code>./run.sh status</code>	Show container status

14. Troubleshooting

Postgres connection refused

Ensure the Docker container is running: `docker ps | grep estimation_postgres` . If not, re-run `./dev.sh infra` . Confirm `POSTGRES_PORT=5436` in `backend/.env.local` .

ChromaDB not found

Check `docker ps | grep estimation_chroma` . Confirm `CHROMA_HOST=localhost` and `CHROMA_PORT=8080` in `backend/.env.local` .

alembic upgrade head fails

Make sure Postgres is running and `backend/.env.local` credentials match the Docker compose values.

BSR ingest already done, re-run needed

Pass the `--force` flag:

```
python scripts/ingest_bsr.py --force
```

Frontend can't reach backend

Check that `NEXT_PUBLIC_API_BASE_URL=http://localhost:8000` is set in `frontend/.env.local` and the backend is running.

Email not sending

Use a Gmail **App Password**, not your regular password. Ensure 2-Step Verification is enabled on your Google account.

Port already in use

Change the conflicting port in `backend/.env.local` (e.g. `POSTGRES_PORT`) and restart the Docker containers.